

Champion Document

Name: __Gagan

Feature: __Summon System & Persistence

Mark: _____ / 30

How this document is graded (30 points)

Criterion	Points	What it means
AI question log	5	The complete log of every question you asked the AI while producing this document, submitted with it. The log shows interrogation, not dictation: verification questions, corrections, and cuts — not one prompt and an export
Internal consistency	10	The five sections describe the same feature: exceptions branch from numbered basic-sequence steps; DFD processes match the use cases; every requirement your feature covers maps to a test with mechanical pass/fail bounds (no adjective where a number should be); timeline tasks match the work described
Team consistency	10	Assessed against your teammates' Champions as a set: the combined documents tile the whole RFP with no overlaps and no gaps; boundaries follow GRASP (each feature owns the data it needs, low coupling, high cohesion) so no one is blocked waiting on a teammate; critical-path items are scheduled first, so if a developer drops mid-semester the product as a whole still ships
Feasibility & adoption	5	Hours are credible for your skill; the critical path is visible; interfaces to teammates' features are named. Where the AI produced the estimates, you have checked them against your own speed and adopted them as your personal commitment — these numbers become the team Gantt and variance is tracked against them

Submit your AI question log with this document, and end the document with a short log summary:

1. Brief Introduction

My feature is the Summon System and Persistence System. A summon is an AI-controlled helper that assists the player automatically during a run. The system supports four summon behaviors: Orbiting, Hunting, Totem, and Revenge. Each behavior is implemented as a plain C# subclass of an abstract Summon class, so the correct UpdateBehavior method is selected at runtime through dynamic binding.

When TL1 starts a run, my SummonFacade loads the player's unlocked summon data and creates the active helper. The helper reads player information from TL6 and valid targets from TL3. When it is ready to attack, it sends a typed AttackRequest to TL2. It does not calculate weapon damage itself. The

system also uses the Memento pattern to capture summon unlocks and permanent summon bonuses so they can be restored after the game closes.

The feature deliberately does not control player movement, enemy AI, weapon damage, XP/loot rewards, dating dialogue, or the overall game state. TL5 awards an unlock; TL2+ validates and saves it. TL1 announces run start/end; TL2+ creates or cleans up summons. This keeps the system focused on summon behavior and summon persistence while avoiding overlap with teammates.

Design structure

Dynamic binding: Summon -> OrbitingSummon / HuntingSummon / TotemSummon / RevengeSummon.

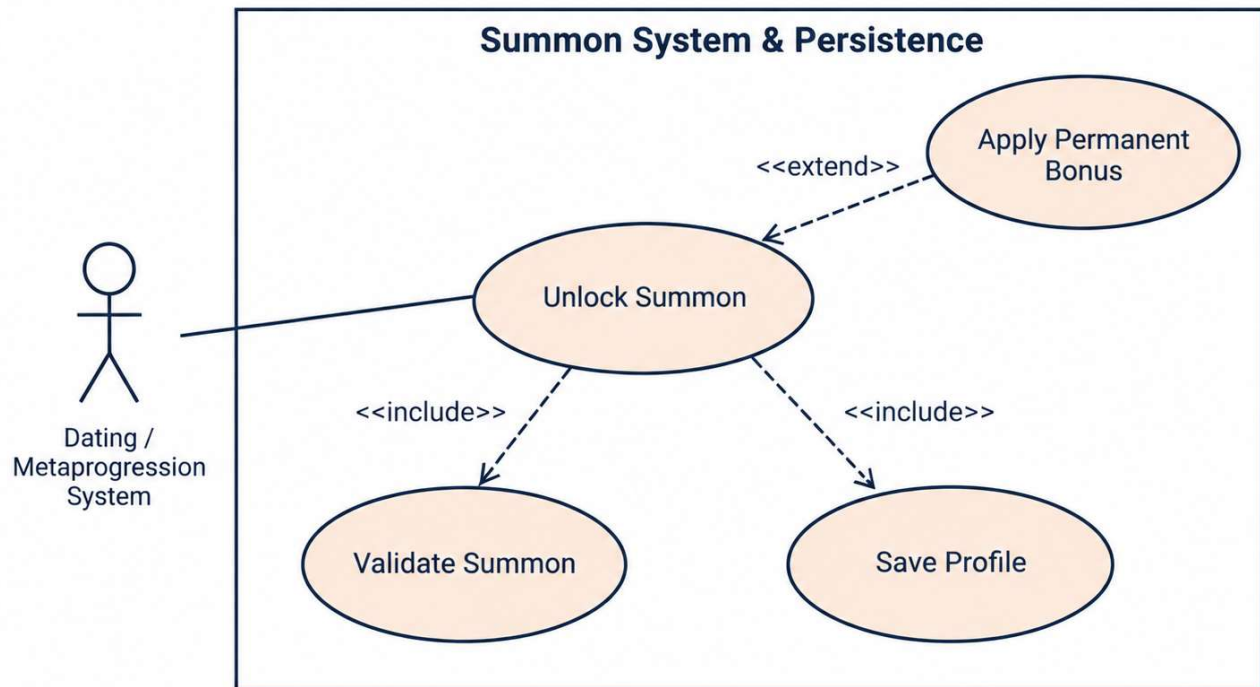
Memento: PlayerProfile is the Originator, ProfileMemento stores a snapshot, and SaveRepository is the Caretaker.

Facade: SummonFacade gives teammates the small public surface UnlockSummon, InitializeRun, UpdateSummons, and EndRun.

The Summon hierarchy is plain C#. A separate Unity-facing SummonController may inherit MonoBehaviour and own the active Summon object.

2. Use Case Diagram with Scenario

Use Case 1 - Unlock and Persist Summon



Scenario 1

- **Name:** Unlock and Persist Summon
- **Summary:** The Dating/Metaprogression System awards a summon; TL2+ validates it, updates the player profile, and saves the new state
- **Actors:** Dating / Metaprogression System (TL5)
- **Preconditions:** The player profile and summon definitions are loaded; the persistence service is available
- **Post conditions:** A successful summon ID exists exactly once in the persistent profile and is available in the next session.
- **Priority:** 1 - Must have
- **ID:** SU1
- **Basic sequence**
 - Step 1: TL5 sends UnlockSummon(summonId).
 - Step 2: SummonFacade confirms that the summon ID exists in Summon Definitions.
 - Step 3: PlayerProfile checks whether the summon is already unlocked.
 - Step 4: The valid ID is added to the player's unlocked-summons set.

Step 5: If the definition contains a permanent bonus, that bonus is applied to the profile.

Step 6: PlayerProfile creates a ProfileMemento containing the updated summon state.

Step 7: SaveRepository saves the Memento.

Step 8: SummonFacade returns UnlockSucceeded with the summon ID.

- **Exceptions**

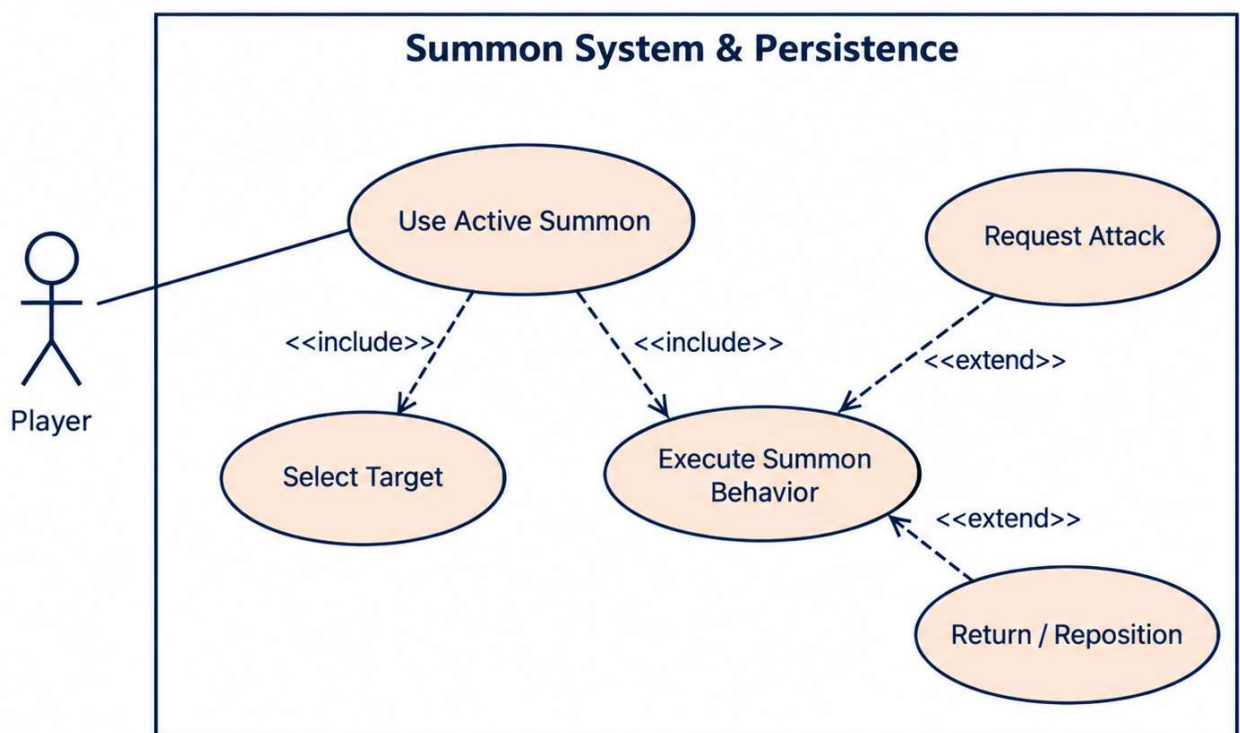
Step 2a: The ID does not exist. Reject the request, change no profile data, and return InvalidSummonId.

Step 3a: The summon is already unlocked. Add no duplicate and return AlreadyUnlocked.

Step 5a: The summon has no permanent bonus. Skip Step 5 and continue at Step 6.

Step 7a: Saving fails. Restore/retain the last valid Memento and return SaveFailed instead of success.

Use Case 2 - Run an Autonomous Summon



- **Name:** Run an Autonomous Summon
- **Summary:** During an active run, the selected summon executes its subclass behavior and assists the player automatically.
- **Actors:** Player

- **Preconditions:** A run is active; an unlocked summon, player context, and enemy-target interface are available.
- **Post conditions:** The summon is active, safely idle, or cleaned up; it does not change data owned by another feature.
- **Priority:** 1 - Must have
- **ID:** SA1
- **Basic sequence**

Step 1: TL1 changes the game state to RUN.

Step 2: SummonFacade initializes the active summon from the player's persisted unlock data.

Step 3: The system receives player position/state and the current valid enemy targets.

Step 4: A base Summon reference calls UpdateBehavior(context); dynamic binding selects the subclass override.

Step 5: The summon selects a target or destination appropriate to its subclass.

Step 6: The summon moves, or holds its required position.

Step 7: When its attack condition is true, it sends a typed AttackRequest to TL2.

Step 8: Steps 3-7 repeat while the run and summon remain active.

- **Exceptions**

Step 2a: No valid summon is available. Create no summon and allow the run to continue.

Step 3a: No enemy target exists. Execute non-combat behavior without sending an attack.

Step 5a: A selected enemy becomes invalid. Clear it and resume target selection at Step 3.

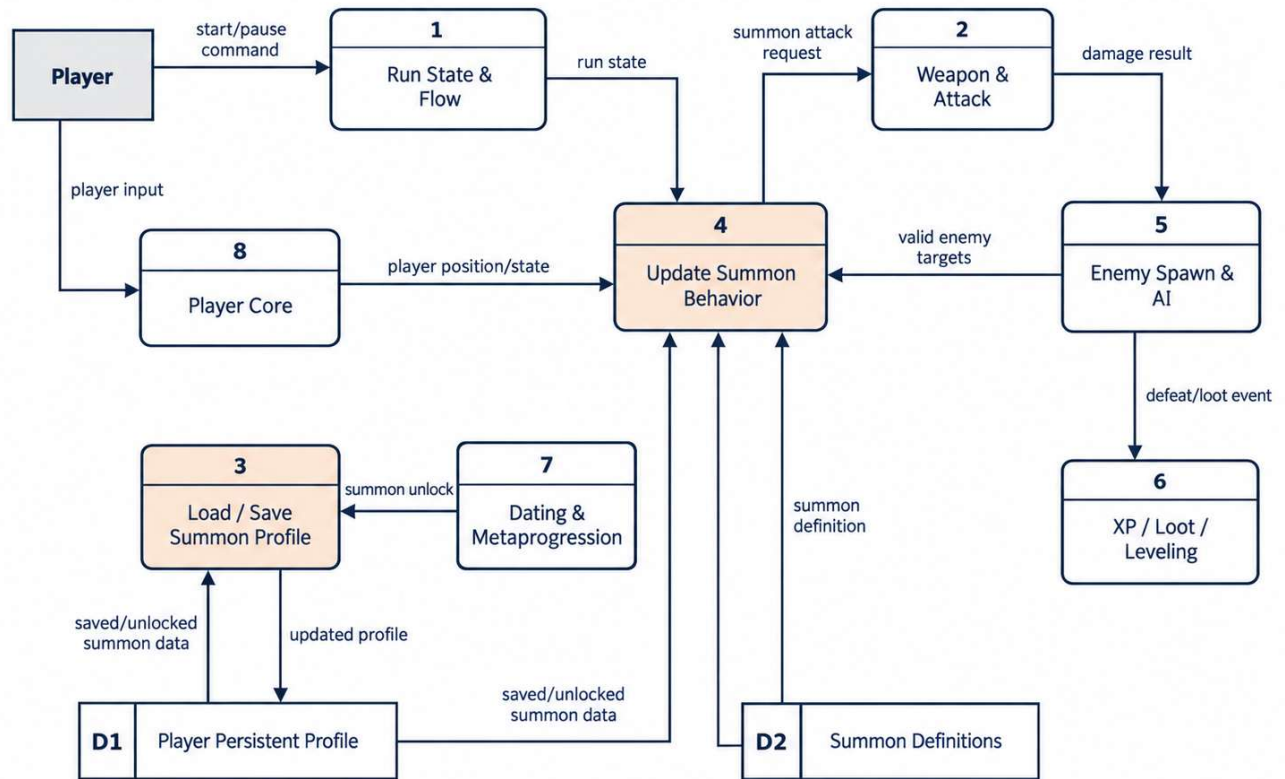
Step 7a: TL2 rejects an invalid target. Apply no damage and return to Step 3.

Step 8a: TL1 sends RunEnded. Stop updates, unregister, and remove every runtime summon object.

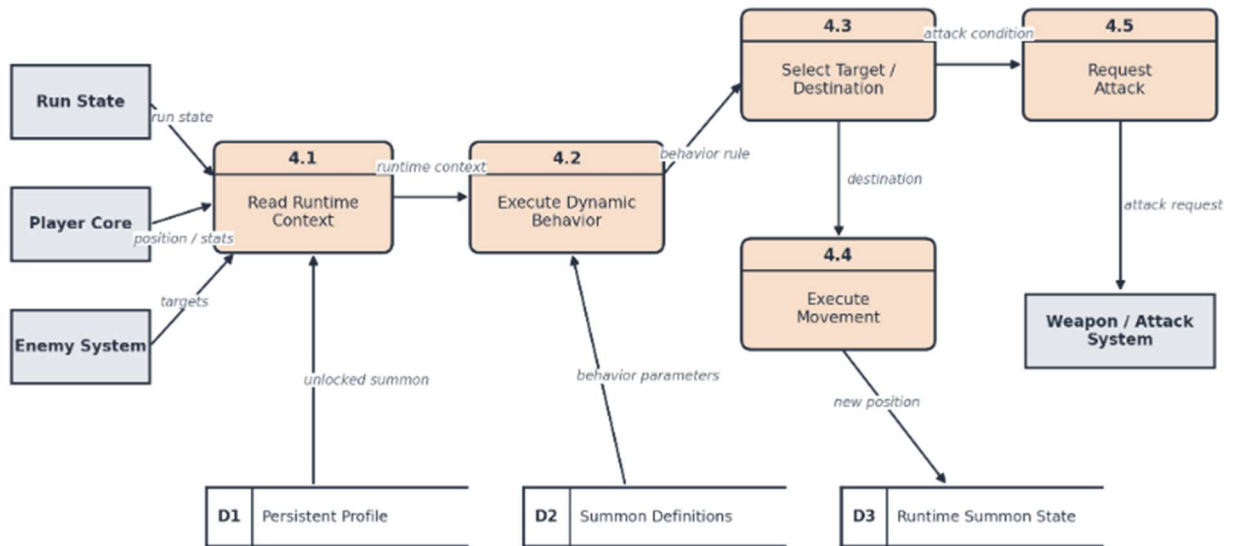
3. Data Flow Diagrams

Diagram 0 shows the whole game and highlights TL2+'s two processes. The feature zoom then decomposes Process 4. Every process has at least one input and one output; persistent and configuration data come from named stores.

Diagram 0 — Whole Game (summon feature processes highlighted)

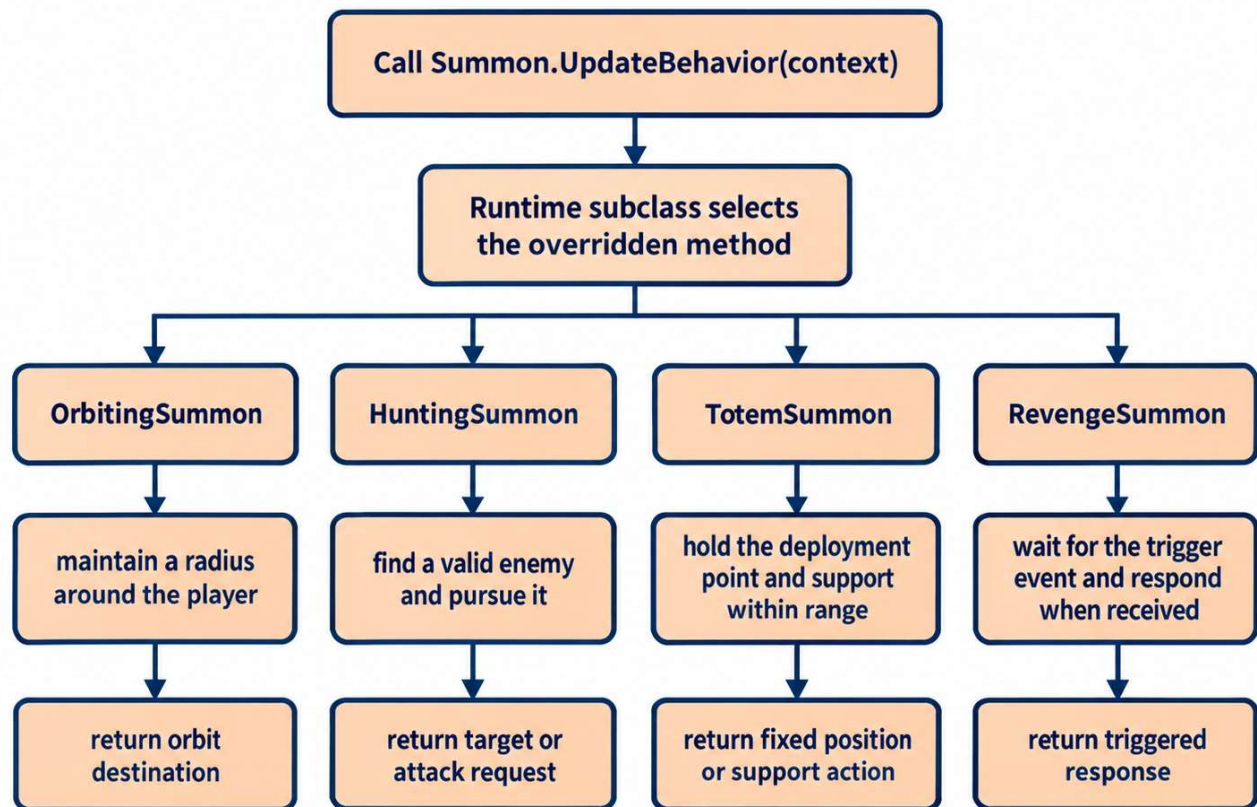


Feature Zoom - Process 4: Update Summon Behavior



Process Description - 4.2 Execute Dynamic Summon Behavior

Process Description — 4.2 Execute Dynamic Summon Behavior



4. Acceptance Tests

The tests below use deterministic test doubles for target data and save failures. Random selections use fixed seeds so any failure can be reproduced on the testing compile path.

Test 1 - Persistence

Run 200 save/load cycles: 50 for each summon type. Pass only if 200/200 reload the exact expected IDs, with 0 missing, 0 extra, and 0 duplicates.

Test 2 - Duplicate Unlock

Send the same unlock request 100 times. Pass only if the profile contains exactly one instance of that summon ID and remains readable.

Test 3 - Dynamic Binding

Instantiate each subclass 25 times through a base Summon reference (100 calls). Pass only if 25/25 calls per type execute the matching override with no external cast or type-switch.

Test 4 - Invalid Target

In 100 cases, invalidate the target immediately before attack. Pass only if 0 attacks reach destroyed targets, 100/100 cases clear or replace the target, and 0 unhandled exceptions occur.

Test 5 - Run Cleanup

Start and stop 100 runs with active summons. Pass only if every run ends with 0 registered runtime summons, 0 post-RunEnded updates, and unchanged persistent unlock data.

Test 6 - Save Failure / Memento

Force 50 save failures. Pass only if 50/50 attempts retain or restore the last valid profile and 0 profiles become partially updated or unreadable.

Requirements-to-tests traceability

Requirement	Acceptance test(s)
Unlocks survive future sessions	Test 1, Test 2, Test 6
Four summon subclasses behave polymorphically	Test 3
Runtime summon safely handles target changes	Test 4
Run lifecycle creates and cleans runtime helpers	Test 5
Failed persistence does not corrupt the profile	Test 6

Sample Output / Analysis

Test	Runs	Observed	Required bound	Pass?
Persistence	200	200 exact reloads	200/200	Yes
Duplicate unlock	100	1 stored ID	Exactly 1	Yes
Dynamic binding	100	100 correct overrides	100/100	Yes
Invalid target	100	2 invalid attacks	0 allowed	No
Run cleanup	100	0 leftover objects	0 allowed	Yes
Failed-save recovery	50	49 restored	50/50	No

5. Timeline

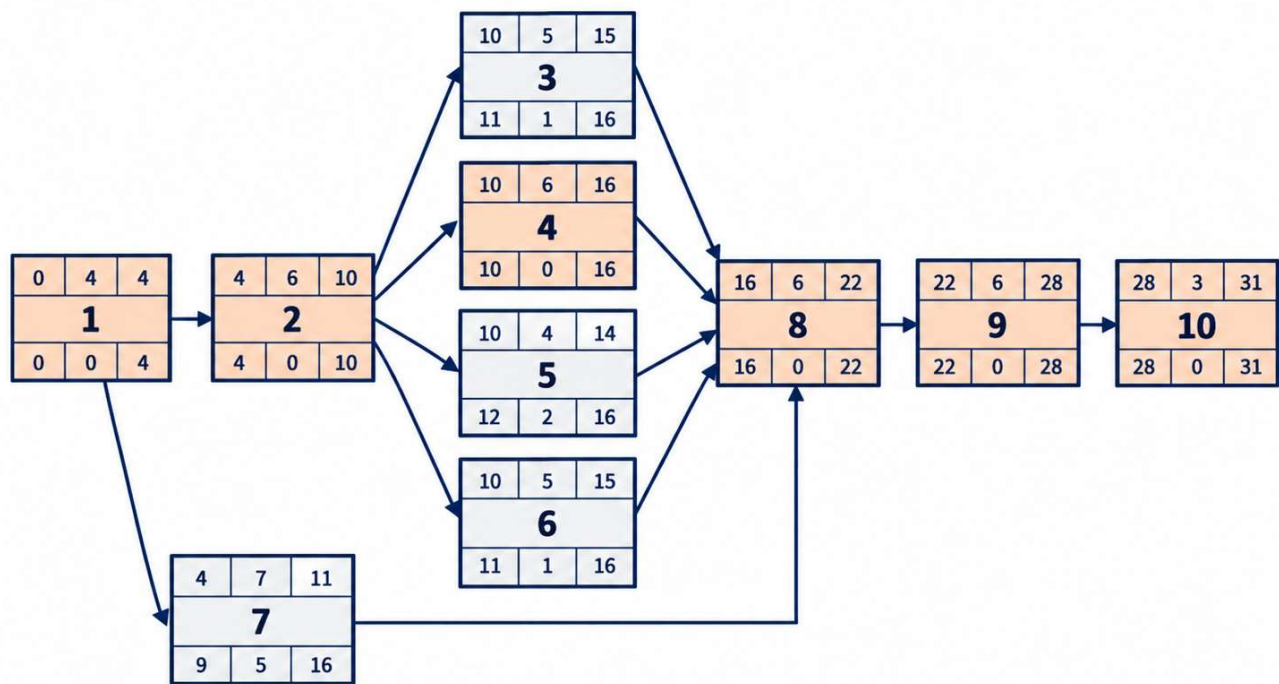
The work totals 52 hours. The dependency-constrained critical path is 31 project hours because several summon behaviors and persistence work can proceed in parallel after the shared foundations are complete.

Task	Duration	Predecessor task(s)
1. Summon data contracts & definitions	4h	-

2. Base Summon hierarchy + Façade	6h	1
3. Orbiting summon behavior	5h	2
4. Hunting summon behavior	6h	2
5. Totem summon behavior	4h	2
6. Revenge summon behavior	5h	2
7. Profile persistence + Memento	7h	1
8. Team-feature integration	6h	3,4,5,6,7
9. Automated testing & debugging	6h	8
10. Documentation & handoff	3h	9

PERT diagram

[



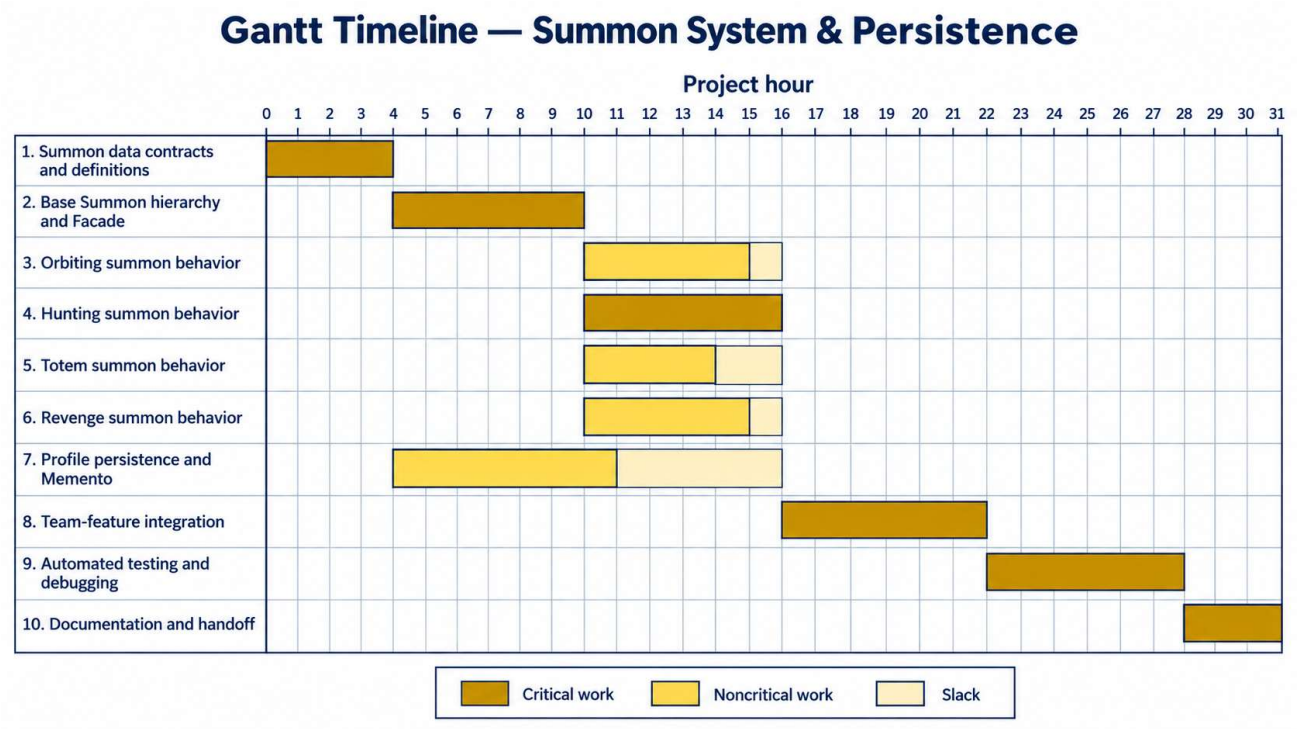
Critical path: 1 → 2 → 4 → 8 → 9 → 10

Total project duration: 31 hours.

Critical path: 1 -> 2 -> 4 -> 8 -> 9 -> 10 = 31 project hours. Tasks 3 and 6 each have 1 hour of slack, Task 5 has 2 hours, and Task 7 has 5 hours.

1w

Gantt timeline



AI Question Log – Summary

I used ChatGPT to help create and format the diagrams, tables, PERT diagram, and Gantt chart. I asked questions to verify the UML arrows, DFD symbols, process names, task dependencies, calculations, and critical path. I reviewed the results and used only the versions that matched my assigned Summon System feature; the written explanations in this document are my own. The complete list of my questions and the full conversation are saved in a separate ChatGPT chat, which I am submitting with this document.